

UNIwersytet Jagielloński
Wydział Matematyki i Informatyki
Instytut Informatyki Analitycznej

Karol Bielaszka
Numer albumu: 1203964

IMPLEMENTACJA ALGORYTMU PRZEBICIA BARIERY
LOGARYTMICZNEJ W PROBLEMIE SZUKANIA
NAJKRÓTSZYCH ŚCIEŻEK W GRAFACH WĄŻONYCH

Na podstawie pracy: Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, Longhui Yin,
Breaking the Sorting Barrier for Directed Single-Source Shortest Paths.

Repozytorium z implementacją:
[https://github.com/Loloekk/Breaking-the-Sorting-Barrier-for-SSSP—Implementation.](https://github.com/Loloekk/Breaking-the-Sorting-Barrier-for-SSSP—Implementation)

Praca licencjacka
na kierunku: Informatyka Analityczna

Praca wykonana pod kierunkiem:
dr Lech Duraj

Kraków, 2026

Spis treści

Wstęp	3
1 Działanie algorytmu	5
1.1 Założenia	5
1.2 Pomysł	6
1.3 Funkcja FindPivots	6
1.4 Funkcja BMSSP	7
1.5 Działanie struktury D	8
1.6 Funkcja BaseCase	8
2 Implementacja struktur	9
2.1 BatchPriorityQueue	9
2.1.1 Erase	10
2.1.2 Insert	10
2.1.3 BatchPrepend	10
2.1.4 Pull	11
2.1.5 Konstruktor	11
2.1.6 Destruktor	11
2.2 LinkedList	11
2.3 IndexedPriorityQueue	12
2.4 Mediana Median	12
3 Implementacja algorytmu	13
3.1 Init	13
3.2 Solve	13
3.2.1 Funkcja BMSSP	13
3.2.2 Funkcja BaseCase	14
3.2.3 Funkcja FindPivots	14
3.3 Get	14
3.4 Free	15

4	Testy	16
4.1	Testy unitarne	16
4.2	Generowanie testów poprawnościowych i wydajnościowych	16
4.2.1	BinomialRandomGraphGenerator	17
4.2.2	RandomTreeGenerator	19
4.2.3	HighRandomGraphGenerator	20
4.2.4	RandomGraphGenerator	21
4.3	Podsumowanie testów wydajnościowych	24
5	Podsumowanie	25
6	Uruchamianie algorytmu i testów	25

Wstęp

Problem szukania najkrótszych ścieżek w skierowanych grafach ważonych jest fundamentalnym pojęciem w informatyce i teorii grafów. Domyślnie zakładamy, że szukamy najkrótszych ścieżek do wszystkich wierzchołków grafu wychodzących z jednego, wyróżnionego wierzchołka początkowego. Zakładamy również, że wagi krawędzi w grafie są nieujemne.

Zastosowanie tego algorytmu jest bardzo szerokie. Jesteśmy w stanie wykorzystać je w każdym problemie, który można zareprezentować za pomocą grafu. Z praktycznych przykładów można wymienić: znajdowanie najkrótszej trasy w nawigacji samochodowej, znajdowanie najszybszego routingu w przesyłaniu danych przez internet, czy rozwiązywanie problemów logistycznych i optymalizacyjnych.

Podstawowym algorytmem do rozwiązywania tego problemu jest algorytm Dijkstry, opracowany w 1959 roku przez Edsgera W. Dijkstrę. Algorytm ten działa w złożoności czasowej $O(m \log n)$, gdzie m oznacza liczbę krawędzi, a n liczbę wierzchołków grafu. Czynniki $\log n$ wynika bezpośrednio z konieczności wielokrotnego wyciągania wierzchołka o najmniejszej wadze z kolejki priorytetowej i aktualizowaniu wagi każdej krawędzi. Z tego powodu operacja ta wymusza sortowanie przetwarzanych wierzchołków. W 1984 roku Fredman i Tarjan wprowadzili kopce Fibonacciego, dzięki którym aktualizacja wagi krawędzi w kopcu działa w czasie amortyzowanym $O(1)$. W ten sposób osiągnięto poprawę złożoności algorytmu Dijkstry do $O(m + n \log n)$. Przez wiele lat w środowisku naukowym utrzymywało się silne przekonanie, że logarytm w złożoności, czyli „bariera sortowania”, jest nieprzekraczalna, a sam ten algorytm stanowi optymalne rozwiązanie.

W kwietniu 2025 badacze Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu oraz Longhui Yin opublikowali pracę pt. „*Breaking the Sorting Barrier for Directed Single-Source Shortest Paths*” (arXiv:2504.17033), w której zaprezentowali pierwszy deterministyczny algorytm rozwiązujący problem SSSP w czasie $O(m \log^{2/3} n)$. Wynik ten przełamał barierę sortowania. Głównym pomysłem umożliwiającym poprawę złożoności było odejście od bezpośredniego utrzymywania porządku wierzchołków. Zamiast pełnego sortowania elementów na granicy poszukiwań, algorytm łączy koncepcje relaksacji znane z algorytmu Bellmana-Forda z rekurencyjną techniką częściowego porządkowania. Pozwala to na zre-

dukowanie liczby operacji porównywania. Warto zauważyć, że dla grafów gęstych przedstawiany algorytm będzie wolniejszy od algorytmu Dijkstry.

Niniejsza praca przedstawia intuicyjne wyjaśnienie algorytmu, implementację oraz porównanie wydajności z algorytmem Dijkstry dla różnych typów grafów.

Rozdział 1

Działanie algorytmu

1.1 Założenia

Sam algorytm stawia dwa mocne założenia. Po pierwsze, każdy wierzchołek w grafie ma stopnie wejściowy i wyjściowy co najwyżej równe 2. Po drugie, każda ścieżka (wyznaczona przez ciąg kolejnych krawędzi) ma inną długość.

Aby uzyskać maksymalne stopnie wchodzący i wychodzący co najwyżej równe 2, rozbijamy każdy wierzchołek na cykl. W każdym cyklu wierzchołki są połączone krawędziami o wadze 0. Natomiast krawędzie wychodzące i wchodzące do oryginalnego wierzchołka są podłączane do kolejnych wierzchołków na odpowiednim cyklu.

O wiele trudniejsze jest zagwarantowanie różnych długości ścieżek. W tym celu jako odległość wierzchołka definiujemy krotkę $(d, \ell, v_\ell, v_{\ell-1}, \dots, v_1)$, gdzie d oznacza ważoną długość ścieżki, ℓ oznacza liczbę wierzchołków na ścieżce, natomiast ciąg $v_1, \dots, v_\ell$ oznacza kolejne wierzchołki na ścieżce. Tak więc v_1 to źródło, natomiast v_ℓ to wierzchołek, do którego prowadzi dana ścieżka. Tak zdefiniowane odległości porównujemy leksyko-graficznie. Jasno widać, że w ten sposób dwie krawędzie nie mogą być tej samej długości (domyślnie zakładamy, że nie ma multikrawędzi). Może to być jednak dla nas problem w sytuacji, gdy różnica w wierzchołkach następowałaby dopiero po rozważeniu wielu krawędzi, ponieważ algorytm mógłby stracić złożoność. Będziemy zatem utrzymywać tylko pierwsze 4 elementy zdefiniowanej krotki. Okazuje się, że ten sposób na potrzeby rozważanego algorytmu jest wystarczający. Dzięki temu porównanie długości wykonujemy w czasie stałym. Możemy tak zrobić, ponieważ jeżeli relaksujemy krawędź z wierzchołka u do wierzchołka v i odległość do wierzchołka v jest taka sama jak dotychczas policzona odległość (porównując pierwsze 4 elementy krotki), to oznacza, że relaksacja poprawiła odległość wierzchołka. Zauważmy, że skoro relaksujemy krawędź wychodzącą z wierzchołka u , to znaczy, że jemu wcześniej poprawiła się odległość, zatem odległość ścieżki idącej jedną krawędź dalej też będzie lepsza niż poprzednia.

Jako wierzchołki kompletne będziemy uznawać te wierzchołki, których wyliczona odległość jest już ostateczna. Wierzchołki niekompletne to te, których odległość musimy jeszcze poprawić. Przyjmijmy stałe $k := \lfloor \log^{1/3}(n) \rfloor$ oraz $t := \lfloor \log^{2/3}(n) \rfloor$.

1.2 Pomysł

Główny pomysł na zarządzanie wykonywanymi operacjami przez algorytm to podział całej pracy na mniejsze części i egzekwowanie ich za pomocą ograniczeń odległości.

Nasz algorytm jest rekurencyjny. Aby go dobrze zrozumieć, załóżmy, że funkcja $\text{BMSSP}(1, B, S)$ to magiczna skrzynka. Jej argumenty oznaczają odpowiednio: poziom rekurencji, ograniczenie górne długości oraz zbiór wierzchołków, z których jesteśmy w stanie wyliczyć wszystkie ostateczne odległości do wierzchołków, których odległość jest mniejsza niż B . Dodatkowo mamy ograniczenie, że $|S| \leq 2^{lt}$. Zatem parametr l mówi nam, z jak dużym podproblemem mamy do czynienia. Parametr B określa nam, jak daleko mamy wykonywać swoją pracę, natomiast S dostarcza nam wierzchołki, z których możemy wyliczyć nasze zadanie. Co więcej, ograniczenie B spełnia również ważną rolę, ponieważ nie mamy gwarancji, że rekurencyjne relaksowanie wszystkich odległości wychodzących z wierzchołków ze zbioru S wyliczy nam wszystkie odległości poprawnie. Zatem ograniczenie B zapobiega nam liczeniu odległości do wierzchołków, do których nie mamy gwarancji, że uda nam się je wyliczyć.

Wynikiem działania magicznej skrzynki są dwie rzeczy: nowe ograniczenie górne B' (gdzie $B' \leq B$) oraz zbiór U , zawierający wierzchołki z ostatecznie wyliczonymi odległościami. Zwracamy ograniczenie B' , ponieważ czasami zagęszczenie krawędzi może być tak duże, że zbiór U przekroczyłby ograniczenie $k2^{lt}$, co zepsułoby złożoność. W takiej sytuacji algorytm przerywa pracę przedwcześnie i zwraca faktyczną granicę B' , do której wyliczył wszystkie odległości. Nasza magiczna skrzynka działa w złożoności $O((kl + tl/k + t)|U|)$, gdzie U to zbiór wierzchołków, których odległości wyliczamy.

1.3 Funkcja FindPivots

Na początku nasza magiczna skrzynka otrzymuje zbiór wierzchołków startowych S . Aby nie przetwarzać wszystkich bezmyślnie, chcemy wybrać tylko te, z których istnieją długie, najkrótsze (pod względem wag) ścieżki. W tym celu stosujemy funkcję `FindPivots`. Relaksujemy wszystkie wierzchołki ze zbioru S przez dokładnie k kroków w głąb i sprawdzamy, które wierzchołki tworzą drzewa wielkości co najmniej k . Korzenie tych drzew uznajemy za pivots i to z nich będziemy puszczać dalszą część naszego algorytmu. Tutaj też musimy uważać, aby relaksacja nie zajęła za dużo czasu. Zatem w momencie, gdy zrelaksujemy więcej niż $k|S|$ wierzchołków, przerywamy działanie algorytmu i za

zbiór pivotów uznajemy cały zbiór S . Natomiast co z wierzchołkami, które nie są wyliczalne ze zbioru pivotów? Oznacza to, że nie tworzą drzew wielkości co najmniej k , zatem ich najkrótsze ścieżki zostały wyliczone wykonując relaksacje w głąb. Zapamiętujemy je w zbiorze W (tak naprawdę zapamiętujemy tu wszystkie wierzchołki, do których doszliśmy wykonując relaksacje) i dodamy je jako przetworzone na końcu wywołania magicznej skrzynki. Dodajemy je na końcu, ponieważ tak naprawdę nie wiemy, dla których wierzchołków odległość jest wyliczona, a dowiadujemy się tego po zakończeniu wywołań rekurencyjnych, gdy pozostałe wierzchołki zostały poprawione.

1.4 Funkcja BMSSP

W celu uniknięcia pełnego sortowania jak w algorytmie Dijkstry, tworzymy strukturę danych D , która zamiast wierzchołka z najmniejszą odległością, zwraca zbiór wierzchołków wielkości $M = 2^{(l-1)t}$ o najmniejszych odległościach. Poza zbiorem tych wierzchołków zwraca również dolną granicę odległości wszystkich wierzchołków, które w niej jeszcze pozostały. Struktura ta pozwala nam na pojedyncze dodanie wierzchołka oraz grupowe dodanie wierzchołków, pod warunkiem, że ich odległości są mniejsze niż dolna granica wszystkich wierzchołków pozostałych w strukturze.

Na początku do struktury wrzucamy wyliczone pivoty z funkcji FindPivots.

Następnie algorytm wchodzi w pętlę, która działa, dopóki zbiór obliczonych wierzchołków U nie stanie się zbyt duży lub dopóki struktura D nie stanie się pusta:

1. Pull: Struktura D zwraca nam zbiór S_i wierzchołków o najmniejszych odległościach (paczka ta ma rozmiar odpowiedni dla niższego poziomu rekurencji). Dodatkowo, D zwraca dolną granicę B_i wszystkich elementów, które w niej jeszcze pozostały. Rekurencyjne wywołania nie mogą wyliczyć odległości wierzchołków, których odległość jest większa niż B_i , ponieważ nie mielibyśmy gwarancji, że będzie to najkrótsza odległość.
2. Rekurencja: Wywołujemy naszą magiczną skrzynkę dla mniejszego podproblemu: $\text{BMSSP}(1 - 1, B_i, S_i)$. Algorytm schodzi poziom niżej.
3. Zbiór i relaksacja: Z niższego poziomu wraca do nas limit B'_i oraz nowo wyliczone wierzchołki U_i . Dodajemy je do naszego zbioru U i relaksujemy ich wychodzące krawędzie.
4. Aktualizacja: Jeśli relaksacja krawędzi prowadzi do krótszej trasy, aktualizujemy odległość danego wierzchołka w strukturze D . Jeśli odległość ta mieści się w widełkach przedziału $[B_i, B)$, używamy zwykłej operacji Insert. Jeśli poprawiona odległość jest mniejsza (w przedziale $[B'_i, B_i)$), to dodajemy wierzchołek do osobnego zbioru i na końcu korzystamy z operacji BatchPrepend, wrzucając takie wierzchołki

na sam przód kolejki. Przy sprawdzaniu relaksacji używamy operatora $\leq$ z dwóch powodów. Po pierwsze, przez definicję odległości za pomocą czteroelementowej krotki. Po drugie, jeżeli w niższych wywołaniach rekurencyjnych odległość została poprawiona, ale przekroczyła granicę B_i , to wierzchołek ten nie został zwrócony i krawędzie z niego wychodzące nie zostały zrelaksowane. Zatem powinien trafić do struktury D i zostać zrelaksowany w kolejnych wywołaniach rekurencyjnych. Tutaj korzystamy z faktu, że nie ma dwóch identycznych długości ścieżek i dzięki temu nie zapętlamy się na cyklach o sumie równej 0.

Podsumowując, algorytm uruchamia rekurencyjne wywołania, które wyliczają ostatecznie pewien podzbiór wierzchołków. Następnie przetwarza je, aby zaktualizować swoją strukturę. To znaczy dodać do rozważanego frontu wierzchołki, do których mamy bezpośrednio dostęp z wierzchołków kompletnych. I znowu uruchamiamy rekurencję w celu wyliczenia kolejnego podzbioru. Aby obliczyć wynik uruchamiamy funkcję $\text{BMSSP}(1, \text{INF}, \{s\})$, gdzie s to wierzchołek początkowy, jako ograniczenie podajemy nieskończoność, aby wyliczyć odległość do każdego wierzchołka, a $l := \lceil \frac{\log(n)}{t} \rceil$.

1.5 Działanie struktury D

Domyślnie możemy na nią patrzeć jak na kolejkę priorytetową działającą na blokach wielkości M . Możemy o tym pomyśleć, że sortujemy nie n elementów, tylko n/M bloków. W ten sposób możemy zredukować koszt całej struktury do $O(n \log(n/M))$, gdzie n to ilość wrzuconych elementów.

Szczegółowe omówienie struktury będzie połączone z omówieniem implementacji.

1.6 Funkcja BaseCase

Kiedy podzadanie osiąga minimalną wielkość ($|S| = 1$), wykorzystujemy zwykły algorytm Dijkstry. Możemy tak zrobić, ponieważ w każdym wywołaniu BMSSP obliczamy dokładnie $k + 1$ wierzchołków, zatem całkowita złożoność pojedynczego wywołania to $O(k \log k)$. Czyli dla wszystkich wierzchołków łącznie jest to $O(n \log k)$. Korzystamy tutaj z faktu, że każdy wierzchołek ma stopień wychodzący co najwyżej 2, dzięki czemu nie rozważymy nagle bardzo dużej liczby sąsiadów jednego wierzchołka.

Rozdział 2

Implementacja struktur

Całą implementację rozwiązania wzorcowego uzależniłem od zmiennej `SafeMode` przekazywanej w teście. Jeżeli pracujemy w trybie `SafeMode`, kod nie sprawdza błędów. Warunki logiczne są usuwane w trakcie kompilacji. Dzięki temu algorytm powinien działać szybciej.

2.1 BatchPriorityQueue

Czyli w naszym algorytmie struktura D.

Założenia:

Mając do wstawienia co najwyżej N par klucz-wartość, parametr całkowitoliczbowy M oraz ograniczenie górne B na wszystkie występujące wartości, tworzymy strukturę danych z następującymi funkcjami:

Insert: Wstawia parę klucz-wartość w amortyzowanym czasie $O(\max\{1, \log(N/M)\})$.

Jeśli dany klucz już istnieje, aktualizuje jego wartość.

Batch Prepend: Wstawia zbiór L par klucz-wartość takich, że każda wartość w L jest mniejsza niż jakakolwiek inna wartość znajdująca się obecnie w strukturze danych. Amortyzowany czas wykonania tej operacji wynosi $O(|L| \cdot \max\{1, \log(|L|/M)\})$. W przypadku wystąpienia wielu par z tym samym kluczem, zachowywana jest ta o najmniejszej wartości.

Pull: Zwraca podzbiór kluczy S' , dla którego zachodzi $|S'| \leq M$, skojarzonych z $|S'|$ najmniejszymi wartościami, a także ograniczenie górne x , które oddziela zbiór S' od pozostałych wartości w strukturze danych. Amortyzowany czas wykonania to $O(|S'|)$. W szczególności, jeśli w strukturze nie ma już żadnych innych elementów, x powinno wynosić B . W przeciwnym razie x powinno spełniać nierówność $\max(S') < x \leq \min(D)$, gdzie D jest zbiorem elementów pozostających w strukturze danych po wykonaniu operacji wyciągnięcia.

W celu zachowania odpowiednich złożoności utrzymujemy dwie listy list. Rozdzielmy sobie je jako listy zewnętrzne i listy wewnętrzne. Listy zewnętrzne to dwie listy, osobna dla metody Insert - $D1$ i osobna dla metody BatchPrepend $D0$. Listy wewnętrzne to elementy list zewnętrznych. Każda lista wewnętrzna przechowuje co najwyżej M elementów. Ponadto dla dwóch list wewnętrznych d_i, d_j w obrębie jednej listy zewnętrznej utrzymujemy porządek. Jeżeli $i < j$, to dla dowolnych elementów $a \in d_i$ oraz $b \in d_j$ zachodzi $a < b$, zakładając, że nie istnieją dwie wartości równe sobie.

2.1.1 Erase

Musimy umieć usuwać elementy z naszej struktury. Chcemy to robić w amortyzowanym czasie $O(1)$, zatem musimy mieć bezpośredni dostęp do miejsca, w którym znajduje się nasz element. W tym celu dla każdego elementu przechowujemy trójkę (p_i, p_o, L) , oznaczające odpowiednio: wskaźnik na dany element (dokładnie na węzeł listy wewnętrznej), wskaźnik na listę wewnętrzną (na węzeł listy zewnętrznej), wartość logiczną mówiącą do której listy zewnętrznej należy element. W sytuacji, gdy usuwamy jeden element, musimy znać jego węzeł i listę wewnętrzną. Jeżeli jednak opróżnimy listę wewnętrzną, to musimy ją usunąć (za wyjątkiem ostatniej listy wewnętrznej w $D1$), więc musimy znać jej listę zewnętrzną.

2.1.2 Insert

Pracujemy wewnątrz listy $D1$. Musimy wstawić element do odpowiedniej listy wewnętrznej. Wraz z listą wewnętrzną przechowujemy jej upperbound (górne ograniczenie). Aby wybrać odpowiednią listę, wystarczy, że znajdziemy upperbound wstawianego elementu wśród ograniczeń list wewnętrznych. W tym celu utrzymujemy mapę, gdzie klucz to upperbound listy, a wartość to wskaźnik na węzeł danej listy. Oczywiście za pomocą utrzymywanych wskaźników sprawdzamy, czy dany klucz nie znajduje się już w strukturze. Jeżeli się znajduje i ma mniejszą wartość to nic nie robimy, jeżeli ma większą wartość to usuwamy go i standardowo wstawiamy nowy element. Jeżeli po dodaniu elementu wielkość wewnętrznej listy przekroczy M , to rozbijamy ją na dwie listy o $\approx M/2$ elementach. Aby tego dokonać, korzystamy z algorytmu mediany median do znalezienia środkowego elementu, a następnie na jego podstawie dzielimy listę.

2.1.3 BatchPrepend

Tę funkcję zaimplementowałem jako wstawienie wektora. Zgodnie z pracą referencyjną wrzucamy wszystkie informacje do jednej listy wewnętrznej, którą dodajemy na początek listy $D0$. Jeżeli jej rozmiar jest za duży, to dzielimy ją rekurencyjnie. Tutaj łatwiej nam będzie utrzymywać minimum zamiast upperbounda tych list, ponieważ aby

wskazać upperbound musielibyśmy się przeiterować po całej następnej liście wewnętrznej. Jeżeli w naszej liście występują elementy o tym samym kluczu wielokrotnie, lub już wcześniej ten klucz występował w tablicy, to musimy wybrać ten o najmniejszej wartości. Usuwając elementy rozważamy dwie opcje. Jeżeli element należy do tej samej listy wewnętrznej, to usuwamy go bezpośrednio z listy, którą teraz tworzymy. Jeżeli należy do innej, to usuwamy go za pomocą domyślnej funkcji erase. Musimy to rozwiązać w ten sposób, ponieważ usunięcie elementu, który należy do aktualnie budowanej listy, może powodować całkowite opróżnienie tej listy, a więc jej usunięcie. To powoduje problemy, ponieważ ta lista nie została jeszcze dodana do listy zewnętrznej.

2.1.4 Pull

Aby uzyskać najmniejsze M elementów oraz górną granicę, wybieramy z obydwu list zewnętrznych taką liczbę list wewnętrznych, aby ilość elementów z każdej z nich przekroczyła M . W ten sposób wiemy, że w naszym wybranym zbiorze znajduje się najmniejsze $M + 1$ elementów. Za pomocą mediany median wyznaczamy $M + 1$ element, wybieramy elementy mniejsze, a ograniczenie B to wartość $M + 1$ elementu. Aby nie martwić się o przypadki brzegowe, obsługujemy je osobnym warunkiem na początku. Dzięki temu, jeżeli pobieramy elementy tylko z jednej listy wewnętrznej, możemy pobierać dane dopóki jest ich mniej niż M , ponieważ dla dokładnie M wyciągniętych elementów ograniczenie wyciągamy z zapamiętanych dla list wewnętrznych upperboundów lub minimów.

2.1.5 Konstruktor

Aby zachować złożoność, nie możemy w każdej instancji struktury D wywoływać alokacji pamięci dla wskaźników na listy. W tym celu przyjmujemy przez konstruktor referencję do zaalokowanej już, pustej tablicy.

2.1.6 Destruktor

Aby zapewnić puste tablice dla innych wywołań, musimy na końcu wyczyścić naszą tablicę. Aby zrobić to optymalnie, czyścimy tylko te elementy, których klucze faktycznie znajdują się w naszej strukturze.

2.2 LinkedList

Jest to standardowa kolejka podwójnie wiązana, operująca na węzłach utrzymujących kopię przetrzymywanej wartości.

2.3 IndexedPriorityQueue

Potrzebujemy ją wykorzystywać do zminimalizowania liczby wierzchołków w kolejce podczas wykonywania base case algorytmu. Jest to standardowy kopiec z tablicą utrzymującą "wskaźniki" do miejsca w kopcu, pod którym znajduje się dany klucz. Analogicznie jak w BatchPriorityQueue, przyjmujemy referencję na tablicę "wskaźników" i sami ją czyścimy na końcu działania struktury.

2.4 Mediana Median

W celu znalezienia k -tej wartości wykorzystujemy medianę median. Przyjmujemy argument jako referencję do wektora wartości, który musi mieć nadpisany operator $<$. Algorytm wykonujemy w miejscu, przekazując w wywołaniach rekurencyjnych wskaźniki na początek i koniec rozważanego przedziału.

Rozdział 3

Implementacja algorytmu

Algorytm napisałem jako klasę umożliwiającą następujące funkcje: `init`, `solve`, `get` i `free`. `Init` jest odpowiedzialna za obliczenie stałych, przekształcenie grafu i alokację pamięci, `solve` jest odpowiedzialna za obliczenie wyników, `get` zwraca wynik, a `free` dealokuje pamięć.

3.1 Init

Alokujemy pamięć dla wszystkich funkcji i struktur. Ważną uwagę należy zwrócić na to, że w konkretnym momencie funkcja `BMSSP` i struktura `D` mają wiele instancji, ale po dokładnie jednym na każdym poziomie rekurencji. Zatem możemy zarezerwować osobne tablice wskaźników i tablice `visited` dla każdego poziomu rekurencji. Pozostałe tablice są alokowane dokładnie raz. W tym miejscu wykonujemy również zamianę krawędzi na wierzchołki. W celu ułatwienia odzyskiwania wyniku w momencie, gdy zamieniamy wierzchołek v na cykl wierzchołków, to jeden z wierzchołków na cyklu ma etykietę dokładnie v . Aby odzyskać wynik ostateczny, wystarczy wziąć odpowiedni prefiks wyniku uzyskanego przez rozwiązanie.

3.2 Solve

Ustawiamy odległość wierzchołka startowego jako krotka $\{0, 0, s, s\}$ i uruchamiamy funkcję `BMSSP(1, INF, {s})`.

3.2.1 Funkcja BMSSP

W celu utrzymywania zbiorów i unikania duplikatów utrzymujemy tablice booli, które informują nas o tym, czy dany element został już wykorzystany. Dodatkowo usuwam ze struktury D elementy, które zostały już zwrócone przez wywołania rekurencyjne jako kompletne. Poza tym algorytm działa dokładnie tak, jak opisano powyżej.

3.2.2 Funkcja BaseCase

Jest to standardowa dijkstra wyliczająca odległości dokładnie $k + 1$ elementów. Ze względu na to, że dane utrzymujemy na kolejce indeksowanej, nie musimy się martwić, że jakiś wierzchołek wystąpi więcej niż jeden raz na kolejce. Zatem nie potrzebujemy żadnej tablicy visited. Dodatkowo względem pracy zmieniłem to, że element jest dodawany do zbioru na końcu pętli while. Dzięki temu $k + 1$ elementu nigdy nie wrzucamy na wektor utrzymujący nasze odpowiedzi.

3.2.3 Funkcja FindPivots

Funkcja działa niemalże identycznie jak w opisie powyżej. Analogicznie w celu utrzymania zbiorów utrzymujemy tablice booli. Jedynie do wyliczania wielkości drzew stworzonych przez wyliczone odległości można było podejść na wiele sposobów. W swoim rozwiązaniu dla każdego wierzchołka utrzymujemy jego pierwsze dziecko i następnego brata. Dzięki temu w łatwy i szybki sposób możemy konstruować drzewo, znając rodziców każdego wierzchołka. Domyślnie wszystkie wartości są ustawione na -1. Jeżeli rozważamy wierzchołek u , którego rodzicem jest wierzchołek p , to wiemy, że wierzchołek u na pewno nie ma jeszcze wyznaczonego brata, a wierzchołek p może mieć wyznaczone pierwsze dziecko. Zatem teraz jako brata wierzchołka u ustawimy pierwsze dziecko wierzchołka p (być może -1), a jako pierwsze dziecko p ustawimy u . Jesteśmy w stanie łatwo iterować się po całym drzewie. Aby rozważyć wszystkie dzieci wierzchołka u , zaczynamy od jego pierwszego dziecka, a do kolejnych przechodzimy za pomocą następnego rodzeństwa. Gdy następne rodzeństwo jest równe -1, to wiemy, że lista dzieci się skończyła. Bardzo ważną kwestią jest to, które wierzchołki rozważamy w naszym grafie. Na początku założyłem, że wszystkie wierzchołki ze zbioru S są korzeniami drzew, przez co nie rozważałem ich jako dzieci ich rodziców. To jest błędem, ponieważ możemy mieć ścieżkę długości k niewychodzącą poza zbiór S , i należy jej korzeń zaliczyć jako pivot. Aby odróżnić korzenie od wierzchołków w środku grafu, sprawdzamy czy ich rodzice znajdują się w zbiorze W . Jeżeli nie, to musi to być korzeń. Czyszczenie tablic odbywa się standardowo na podstawie zbiorów W i S .

3.3 Get

Dzięki temu, że w każdym cyklu wierzchołków znajduje się jeden o takiej etykietce jak oryginalny wierzchołek, to w celu otrzymania ostatecznego wyniku wystarczy przepisać pierwsze n wyników uzyskanych przez algorytm.

3.4 Free

W celu zwolnienia pamięci wykorzystujemy wbudowaną w vector metodę `swap`, która w czasie stałym podmienia pamięć wektorów. Dzięki temu destruktory uruchomione w metodzie `get` automatycznie zwalniają zaalokowaną pamięć, a w pamięci struktury pozostają puste wektory.

Rozdział 4

Testy

4.1 Testy unitarne

Do wszystkich struktur napisałem standardowe testy unitarne sprawdzające poprawność.

4.2 Generowanie testów poprawnościowych i wydajnościowych

Grafy testuję na wielu różnych rodzajach grafów. Domyślnie, jeżeli graf jest nieskierowany, to tak naprawdę jest skierowany, tylko krawędź w obydwu stronach ma taką samą wagę. Wszystkie generatory przyjmują jako argument minimalną i maksymalną wagę krawędzi. Wagi krawędzi są generowane losowo z przekazanego przedziału. Wszystkie generatory mieszają etykiety wierzchołków, aby zapewnić większą losowość testów i tworzyć testy, które domyślnie nie są łatwo cache'owalne.

Dla wszystkich testów tworzę statystyki grafu. Są to następująco: ilość wierzchołków, ilość krawędzi, długość najdłuższej ścieżki ważonej (spośród tych najkrótszych), długość najdłuższej ścieżki nieważonej (spośród tych najkrótszych), długość najdłuższej ścieżki pod względem ilości wierzchołków korzystając jedynie z krawędzi, które tworzą najkrótsze ścieżki w grafie ważonym, minimalna waga krawędzi, maksymalna waga krawędzi, wielkość wierzchołków, do których możemy dotrzeć z wierzchołka startowego, oraz ilość krawędzi, do których możemy dotrzeć z wierzchołka startowego.

W celu porównania czasów działania porównuję teoretyczne rozwiązanie z pracy do zwykłej Dijkstry na standardowej kolejce priorytetowej bez zmniejszania wartości dla klucza. Dla rozwiązywania teoretycznego sprawdzam czasy w trybie SafeMode i bez niego. Jako główny wynik będę traktował czasy osiągnięte przez rozwiązanie z wyłączonym trybem SafeMode. Sprawdzam jedynie czasy działania modułów: init, solve i get.

Dla każdego typu grafu losuję testy 5 razy z dokładnie takimi samymi parametrami, aby móc uśrednić wyniki.

4.2.1 BinomialRandomGraphGenerator

Generator przyjmuje jako argument ułamek, który oznacza prawdopodobieństwo na istnienie każdej krawędzi. Ten generator służy głównie do generowania grafów gęstych.

Grafy Asymetryczne (Gęste)

Nodes count (N): 1 000
Edges count (M): 349 898.6
Longest path length: 46 638 152.8
Longest vertex path: 2.0
Longest vertex path (by weights): 13.8
Minimum edge weight: 2 585.4
Maximum edge weight: 999 996 134.8
Main component size: 1 000
Main component edge count: 349 898.6

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	0.552	5.702	0.010	6.258
Theo Safe	0.588	5.856	0.010	6.452
Dijkstra	0.050	0.020	0.000	0.070

Współczynnik (Theoretical / Dijkstra): 89.40

Grafy Asymetryczne (Rzadkie)

Nodes count (N): 10 000
Edges count (M): 500 187.0
Longest path length: 413 678 675.2
Longest vertex path: 3.2
Longest vertex path (by weights): 20.0
Minimum edge weight: 1 143.2
Maximum edge weight: 999 997 925.4
Main component size: 10 000
Main component edge count: 500 187.0

Współczynnik (Theoretical / Dijkstra): 56.18

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	0.818	8.612	0.010	9.438
Theo Safe	0.894	8.740	0.010	9.644
Dijkstra	0.108	0.060	0.000	0.168

Grafy Symetryczne (Gęste)

Nodes count (N): 1 000
 Edges count (M): 699 315.6
 Longest path length: 25 043 617.2
 Longest vertex path: 2.0
 Longest vertex path (by weights): 13.4
 Minimum edge weight: 1 914.0
 Maximum edge weight: 999 998 233.4
 Main component size: 1 000
 Main component edge count: 699 315.6

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	1.082	12.082	0.010	13.180
Theo Safe	1.126	12.392	0.010	13.530
Dijkstra	0.100	0.030	0.000	0.130

Współczynnik (Theoretical / Dijkstra): 101.38

Grafy Symetryczne (Rzadkie)

Nodes count (N): 10 000
 Edges count (M): 499 757.8
 Longest path length: 405 941 667.4
 Longest vertex path: 3.0
 Longest vertex path (by weights): 20.0
 Minimum edge weight: 1 845.8
 Maximum edge weight: 999 998 229.2
 Main component size: 10 000
 Main component edge count: 499 757.8

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	0.800	8.450	0.010	9.260
Theo Safe	0.852	8.650	0.010	9.512
Dijkstra	0.100	0.060	0.000	0.160

Współczynnik (Theoretical / Dijkstra): 57.88

4.2.2 RandomTreeGenerator

Jest to standardowy generator wysokich drzew, dla każdego wierzchołka losujemy rodzica spośród kilku ostatnich (pod względem etykiety) wierzchołków. W ten sposób tworzą nam się wysokie drzewa.

Drzewa Niskie (N = 1 000 000)

Nodes count (N): 1 000 000
Edges count (M): 1 999 998
Longest path length: 23 115 193 194.0
Longest vertex path: 42.0
Longest vertex path (by weights): 42.0
Minimum edge weight: 1 285.4
Maximum edge weight: 999 998 253.0
Main component size: 1 000 000
Main component edge count: 1 999 998

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	5.274	42.646	0.030	47.952
Theo Safe	5.456	42.782	0.032	48.274
Dijkstra	0.964	1.932	0.000	2.898

Współczynnik (Theoretical / Dijkstra): 16.55

Drzewa Niskie (N = 100 000 000)

Nodes count (N): 100 000 000
Edges count (M): 199 999 998
Longest path length: 31 373 734 168.4
Longest vertex path: 58.8
Longest vertex path (by weights): 58.8
Minimum edge weight: 9.0
Maximum edge weight: 999 999 989.0
Main component size: 100 000 000
Main component edge count: 199 999 998

Współczynnik (Theoretical / Dijkstra): 11.10

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	788.880	5402.638	3.800	6195.318
Theo Safe	773.502	5952.738	4.782	6731.020
Dijkstra	175.036	382.696	0.600	558.330

Drzewa Wysokie (N = 1 000 000)

Nodes count (N): 1 000 000
 Edges count (M): 1 999 998
 Longest path length: 62 548 143 771 331.8
 Longest vertex path: 125 124.2
 Longest vertex path (by weights): 125 124.2
 Minimum edge weight: 1 106.6
 Maximum edge weight: 999 998 689.2
 Main component size: 1 000 000
 Main component edge count: 1 999 998

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	5.356	19.582	0.030	24.974
Theo Safe	5.568	20.130	0.034	25.736
Dijkstra	1.010	0.914	0.020	1.942

Współczynnik (Theoretical / Dijkstra): 12.86

4.2.3 HighRandomGraphGenerator

Generator losuje graf podobnie jak generator wysokich drzew, z tą różnicą, że dla każdego wierzchołka dodaje k połączeń skierowanych losowo. Wierzchołki również są wybierane z kilku najbliższych, jednak niekoniecznie tylko mniejszych etykiet. Dzięki temu generatorowi możemy tworzyć rzadkie grafy z długimi najkrótszymi ścieżkami.

Grafy Asymetryczne

Nodes count (N): 200 000
 Edges count (M): 1 000 000
 Longest path length: 2 254 083 912 696.0
 Longest vertex path: 8 351.8
 Longest vertex path (by weights): 11 896.0
 Minimum edge weight: 1 628.8
 Maximum edge weight: 999 999 330.2

Main component size: 199 457.8

Main component edge count: 995 928.8

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	2.356	11.840	0.022	14.214
Theo Safe	2.462	11.698	0.020	14.174
Dijkstra	0.454	0.312	0.000	0.766

Współczynnik (Theoretical / Dijkstra): 18.56

Grafy Symetryczne

Nodes count (N): 100 000

Edges count (M): 1 000 000

Longest path length: 687 378 443 837.8

Longest vertex path: 4 514.0

Longest vertex path (by weights): 7 210.8

Minimum edge weight: 2 985.2

Maximum edge weight: 999 998 689.8

Main component size: 100 000

Main component edge count: 1 000 000

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	2.090	12.652	0.020	14.758
Theo Safe	2.216	13.030	0.020	15.262
Dijkstra	0.406	0.216	0.000	0.622

Współczynnik (Theoretical / Dijkstra): 23.73

4.2.4 RandomGraphGenerator

W celu optymalizacji rozdzielamy go na dwa przypadki. Gdy graf zawiera mniej krawędzi niż ilość wszystkich możliwych krawędzi podzielona przez dwa, to generator losuje dwa wierzchołki i łączy je krawędzią. Na secie zapamiętujemy, które wierzchołki zostały już połączone, aby uniknąć duplikatów. W przeciwnym wypadku najpierw generujemy wszystkie możliwe krawędzie i spośród nich losujemy ustaloną ilość krawędzi do naszego grafu. Gdybyśmy chcieli wylosować maksymalną możliwą liczbę krawędzi pierwszym sposobem, to zajęłoby to znacznie więcej czasu, ponieważ szansa wylosowania niewygenerowanej jeszcze krawędzi byłaby bardzo mała, więc musielibyśmy losować wielokrotnie dla ostatnich krawędzi.

Grafy Asymetryczne (Gęste)

Nodes count (N): 1 000
Edges count (M): 999 000
Longest path length: 16 659 433.6
Longest vertex path: 1.0
Longest vertex path (by weights): 14.0
Minimum edge weight: 1 139.4
Maximum edge weight: 999 999 050.2
Main component size: 1 000
Main component edge count: 999 000

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	1.466	17.996	0.020	19.474
Theo Safe	1.594	18.154	0.020	19.762
Dijkstra	0.130	0.040	0.000	0.170

Współczynnik (Theoretical / Dijkstra): 114.55

Grafy Asymetryczne (Rzadkie)

Nodes count (N): 200 000
Edges count (M): 1 000 000
Longest path length: 5 016 313 889.6
Longest vertex path: 12.0
Longest vertex path (by weights): 28.6
Minimum edge weight: 793.6
Maximum edge weight: 999 999 007.4
Main component size: 198 606.8
Main component edge count: 993 066.0

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	2.226	18.974	0.020	21.216
Theo Safe	2.364	19.396	0.020	21.778
Dijkstra	0.436	0.616	0.000	1.054

Współczynnik (Theoretical / Dijkstra): 20.13

Grafy Symetryczne (Gęste)

Nodes count (N): 1 000
Edges count (M): 999 000
Longest path length: 15 774 022.0
Longest vertex path: 1.0
Longest vertex path (by weights): 14.6
Minimum edge weight: 1 528.6
Maximum edge weight: 999 996 456.0
Main component size: 1 000
Main component edge count: 999 000

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	1.464	17.722	0.020	19.198
Theo Safe	1.576	17.972	0.020	19.562
Dijkstra	0.130	0.040	0.000	0.170

Współczynnik (Theoretical / Dijkstra): 112.93

Grafy Symetryczne (Rzadkie)

Nodes count (N): 100 000
Edges count (M): 1 000 000
Longest path length: 2 396 143 680.6
Longest vertex path: 7.2
Longest vertex path (by weights): 25.4
Minimum edge weight: 3 748.2
Maximum edge weight: 999 997 830.0
Main component size: 99 995.2
Main component edge count: 1 000 000

Rozwiązanie	Init (s)	Solve (s)	Get (s)	Total (s)
Theoretical	1.994	18.196	0.020	20.204
Theo Safe	2.184	18.830	0.020	21.032
Dijkstra	0.388	0.380	0.000	0.768

Współczynnik (Theoretical / Dijkstra): 26.31

4.3 Podsumowanie testów wydajnościowych

Analizując powyżej uzyskane czasy, łatwo widać, że stała narzucana przez algorytm z pracy jest ogromnie duża. Nie ma większej różnicy w działaniu grafów symetrycznych i asymetrycznych. Możemy również zauważyć nieznaczne przyspieszenie algorytmu w trybie bez SafeMode. Dodatkowo, co było dość spodziewane, wyniki zależą tylko i wyłącznie od liczby krawędzi, a nie wierzchołków. Jednoznacznie wynika to z preprocessingu algorytmu, który dla każdej krawędzi tworzy dwa wierzchołki. Widać, że im większa liczba krawędzi, tym stosunek czasu uzyskanego przez rozwiązanie teoretyczne i Dijkstrę maleje. Jednakże nawet dla drzewa o 10^8 wierzchołkach nadal ten stosunek wynosi około 11. To wbrew pozorom nie jest aż tak duża stała i przy wprowadzeniu większej ilości optymalizacji oraz większym grafie mogłoby dać wyniki, w których rozwiązanie teoretyczne daje wyniki lepsze niż Dijkstra, zakładając oczywiście, że pracowalibyśmy na grafie, który nie jest drzewem, ale jego gęstość jest zbliżona do drzewa.

Rozdział 5

Podsumowanie

Dla większości używanych grafów algorytm ten jest na pewno nieoptymalny. Jednakże jest jeszcze wiele miejsc, w których można by było poprawić jego stałą, a co za tym idzie mógłby okazać się użyteczny dla bardzo dużych grafów. Implementacja algorytmu jest bardzo skomplikowana i jest nieadekwatna w porównaniu do bardzo prostej implementacji Dijkstry.

Rozdział 6

Uruchamianie algorytmu i testów

W celu uruchomienia testów unitarnych należy odpalić komendę `make run_unit_tests`.

W celu uruchomienia testów poprawnościowych (to znaczy małych grafów generowanych jak opisano powyżej) należy uruchomić `make run_correctness_test`.

W celu uruchomienia testów sprawdzających czasy należy uruchomić `make run_speed_test`.

Aby uruchomić algorytm do przetestowania na własnych przykładach, należy uruchomić `make run`. Program ten na wejściu przyjmuje 4 liczby całkowite (n , m , s , d) oznaczające odpowiednio liczbę wierzchołków, liczbę krawędzi, wierzchołek startowy, oraz wartość (0/1) oznaczającą, czy graf jest skierowany (1), czy nieskierowany (0). W kolejnych m wierszach należy podać po trzy liczby całkowite będące opisem kolejnych krawędzi. Wierzchołki powinny mieć etykiety od 1 do n . Wynikiem działania algorytmu jest n linii. i -ta linia zawiera numer wierzchołka i jego odległość od wierzchołka startowego. Na `stderr` wypisywane są statystyki grafu (nie należy pamiętać, że wypisywana liczba krawędzi, to liczba krawędzi skierowanych) oraz czasy działania poszczególnych metod rozwiązania.